2

Unit 2: Gate-Level Minimization (K-Maps)

Digital Logic Design

CHANDRAS EDU

Never stop learning

Outline

2.1 K-Maps: 2 & 3 Variables
2.2 4-Variable K-Maps
2.3 Product-of-Sums (POS) Simplification
2.4 Don’t-Care Conditions
2.5 NAND/NOR Implementation

2.1 K-Maps: 2 & 3 Variables

IN SIMPLE TERMS

A K-map lays out a truth table so that neighbouring cells differ by one variable; circling neighbouring 1s removes that variable.

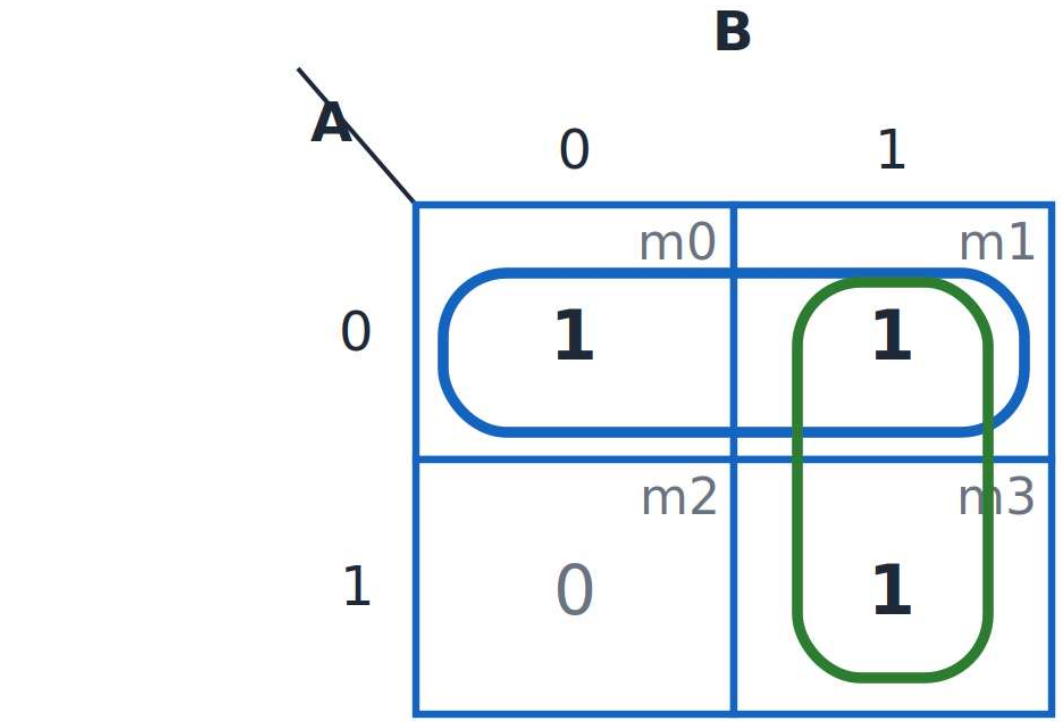
The key idea: adjacency

- Algebraic simplification needs the right theorem at each step, and it is easy to stop too early or make a mistake
- A **K-map** arranges the rows of a truth table in a grid so that **physically adjacent cells differ in exactly one variable**
- Row and column headers are written in **Gray code** order: 00, 01, 11, 10, never 00, 01, 10, 11
- Two cells that differ in exactly one variable combine algebraically: the differing variable cancels ($x + x' = 1$)
- A K-map lets you **see** which cells qualify

2-variable K-map

- A 2-variable map has **4 cells**: one per row of a 2-variable truth table
- The next example is F(A,B) = Σm(0,1,3)
- The figure shows the map with its two groups of adjacent 1s

Two-variable K-map



A′ (top row) **B (right column)**

$$F = A' + B$$

Two-variable K-map for F = Σm(0,1,3): F = A′ + B.

Worked example: F(A,B) = Σm(0,1,3)

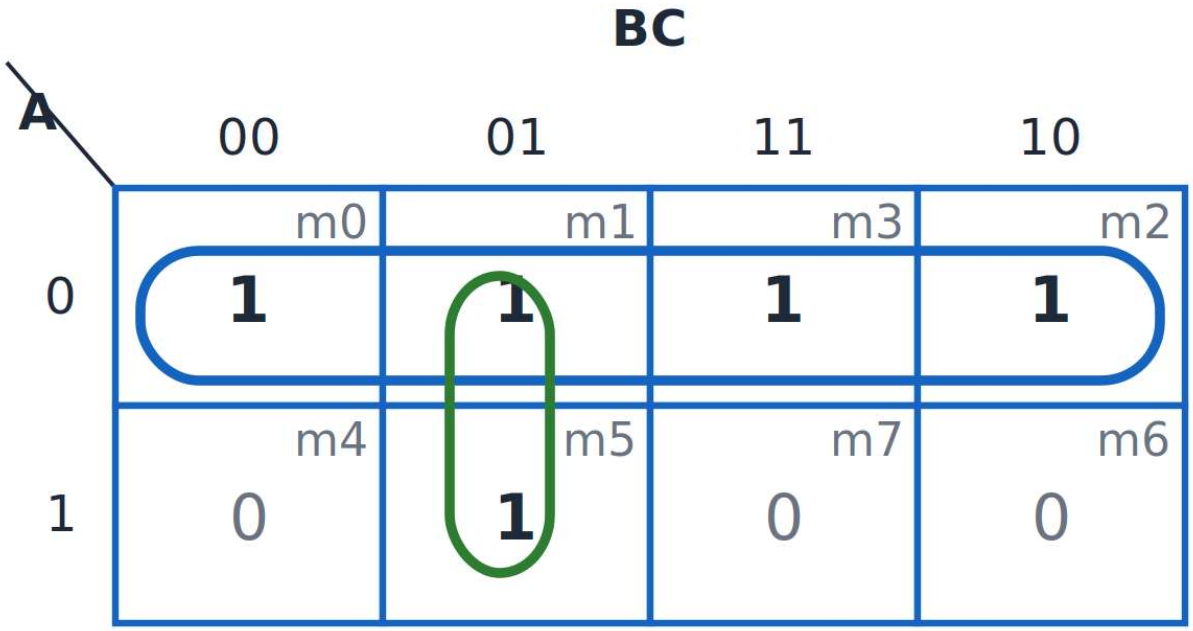

```
Top row (A = 0, B varies):      term = A'
Right column (B = 1, A varies):  term = B
F = A' + B
```

- Each group of size 2 drops one variable, leaving only the variables that stay constant
- Cell m1 belongs to both groups: completely normal
- A group only needs to cover a 1; a 1 may be covered by more than one group

3-variable K-map

- A 3-variable map has **8 cells**: 2 rows (the first variable) by 4 columns
- The column header is in Gray-code order: 00, 01, 11, 10
- The **leftmost and rightmost columns are also adjacent**: the grid wraps around like a cylinder
- The next example is F(A,B,C) = Σm(0,1,2,3,5)

Three-variable K-map



A′ (top row, m0–m3) **B′C (m1, m5)**

F = A′ + B′C

Three-variable K-map for F = Σm(0,1,2,3,5): F = A′ + B′C.

Worked example: F(A,B,C) = Σm(0,1,2,3,5)


```
Top row, all 4 cells (A = 0; B and C vary):    term =
A'
Pair {m1, m5} (B = 0, C = 1; A varies):        term =
B'C
F = A' + B'C
```

- Check against the canonical form: minterms 0, 1, 2, 3 are exactly the A = 0 row (A′)
- Minterm 5 is covered by B′C: together {0,1,2,3,5} matches the target exactly ✓

Grouping rules

- A group's size is always a **power of 2**: 1, 2, 4, 8, … Groups of 3, 5, 6 or 7 cells are not valid
- A group of size 2^n eliminates exactly n variables from the term
- **Every 1** must be covered by at least one group, or the expression is wrong
- **Make groups as large as possible:** a bigger group gives a shorter term
- **Use the fewest groups** that cover every 1; overlap is fine and often necessary
- A group covering the whole map means F = 1

2.2 4-Variable K-Maps

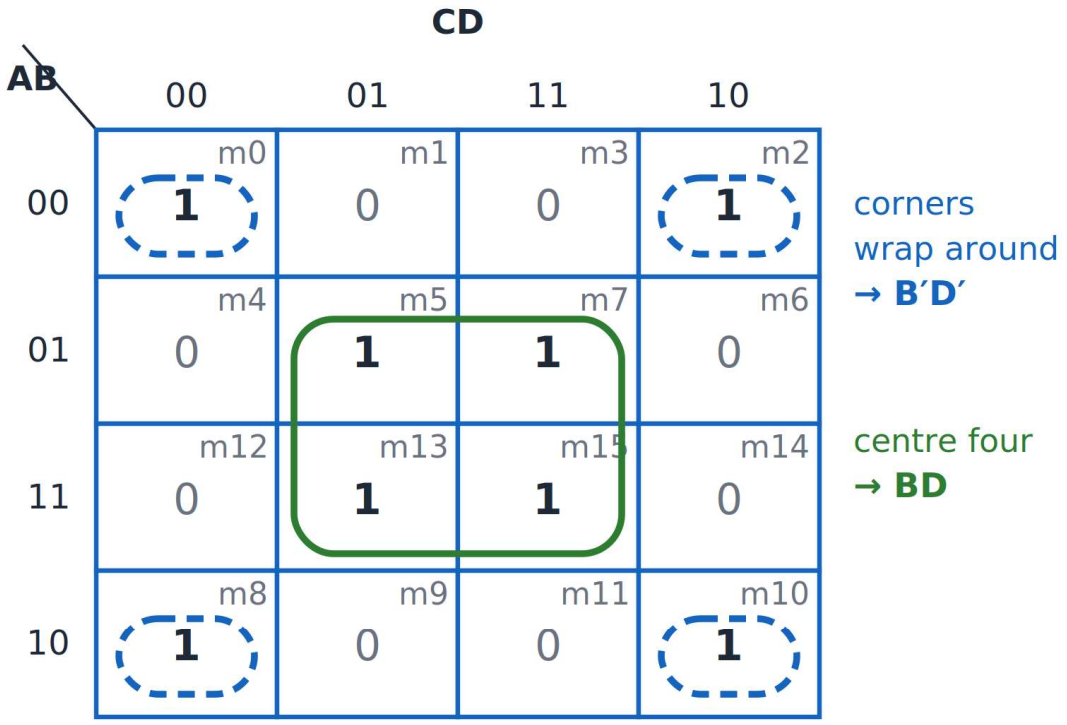
IN SIMPLE TERMS

With four variables the map is a 4 × 4 grid whose edges wrap around: top meets bottom, left meets right, so even the four corners are neighbours.

The grid layout

- A 4-variable map has **16 cells** in a 4 × 4 grid
- Both the rows (AB) and the columns (CD) use Gray-code order: 00, 01, 11, 10
- Rows wrap top-to-bottom (row 00 is adjacent to row 10)
- Columns wrap left-to-right (column 00 is adjacent to column 10)
- The map behaves like the surface of a torus, not a flat rectangle

Four-variable K-map



F = B′D′ + BD

Four-variable K-map: the corners wrap around to give B'D'; the centre four give BD.

Worked example: F = Σm(0,2,5,7,8,10,13,15)


```
Corners {m0, m2, m8, m10}:  B = 0, D = 0  →  B'D'
Quad {m5, m7, m13, m15}:      B = 1, D = 1  →  BD
F = B'D' + BD
```

- In both groups A and C vary, so they drop out
- The four corners are drawn as four separate boxes because a wraparound group cannot be enclosed by one loop on a flat page; they are still one logical group
- All four corners are mutually adjacent: row 00/row 10 wrap vertically and column 00/column 10 wrap horizontally
- Every 1 is covered by exactly one of the two quads, so the result is minimal

Grouping sizes at 4 variables

The pattern from the 3-variable page continues: each doubling of group size eliminates one more variable.

Group size	Variables eliminated	Term length
1 cell	0	4 literals (the full minterm)
2 cells (pair)	1	3 literals
4 cells (quad)	2	2 literals
8 cells (octet)	3	1 literal
16 cells (whole map)	4	F = 1 (constant)

2.3 Product-of-Sums (POS) Simplification

IN SIMPLE TERMS

Circle the 0s instead of the 1s and you get the function as a product of sums.

The flip: group the zeros

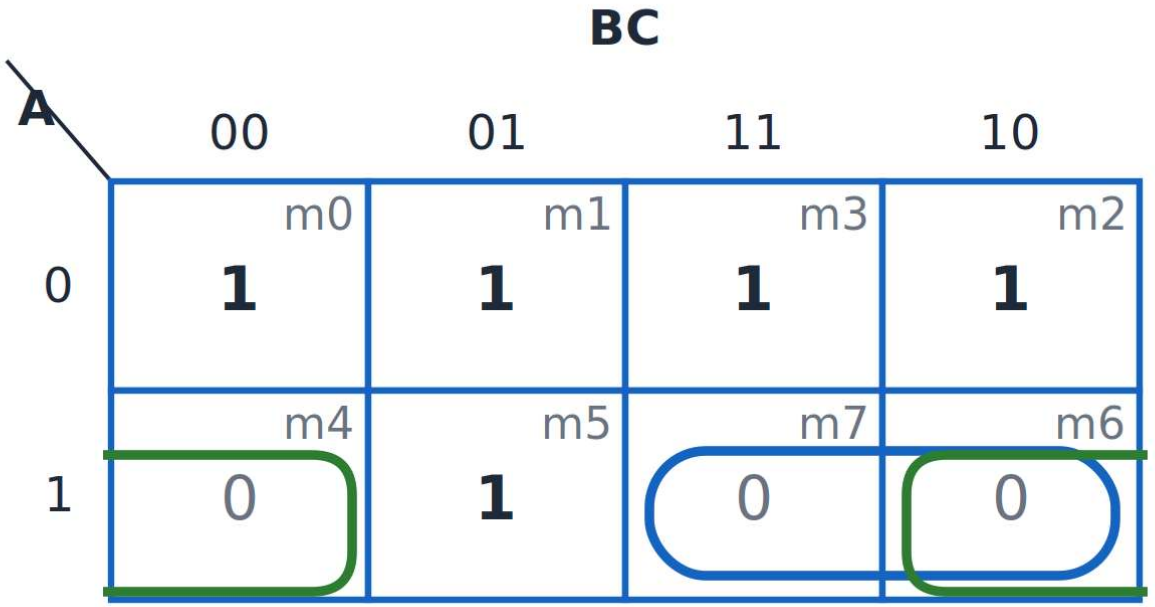
- Apply the same K-map procedure: Gray-code adjacency, power-of-2 groups, as large and as few as possible
- But group the **0-cells** instead of the 1-cells
- Each group gives one **sum (OR) term**, and the final expression **ANDs** those terms together
- The only change is the read-off convention (next slide)

Read-off convention

This mirrors how a maxterm M_i is built from the minterm m_i in Unit 1: the same complementation rule, applied group by group instead of cell by cell.

	Variable = 0 in group	Variable = 1 in group
SOP (grouping 1s)	write complemented (x′)	write uncomplemented (x)
POS (grouping 0s)	write uncomplemented (x)	write complemented (x′)

Grouping the 0s



(A′ + B′) m6, m7

(A′ + C) m4, m6 (wrap)

$$F = (A' + B')(A' + C)$$

Grouping the 0s of F = Σm(0,1,2,3,5): F = (A′ + B′)(A′ + C).

Worked example: F = Σm(0,1,2,3,5), grouping the 0s


```
Pair {m6, m7}:  A = 1, B = 1  →  (A′ + B′)
Pair {m4, m6}:  A = 1, C = 0  →  (A′ + C)
F = (A′ + B′)(A′ + C)
```

- In the first pair C varies; in the second pair B varies
- The three 0-cells cannot all go in one group: m4 and m7 are not adjacent
- m4 and m6 are adjacent because the two ends of a row wrap around

Checking the POS result


```
Test A=0, B=0, C=0 (m0, expect F=1):   (1+1)(1+0) = 1 ✓
Test A=1, B=0, C=0 (m4, expect F=0):   (0+1)(0+0) = 0 ✓
Expand: (A' + B')(A' + C) = A' + B'C   ✓
```

- The expansion is the same function as the SOP result
- F = 0 at exactly {4, 6, 7} and 1 everywhere else, which is what grouping only the 0-cells guarantees

SOP and POS are two realisations

- SOP and POS are two different, equally valid **circuit realisations** of the **same function**
- They are not two different functions
- Which is better depends on the circuit: sometimes SOP uses fewer gates or literals, sometimes POS
- Both are worth computing when gate count genuinely matters

2.4 Don’t-Care Conditions

IN SIMPLE TERMS

A don’t-care is an input that never happens, so you may treat it as 1 or 0, whichever makes the groups bigger.

What a don’t-care is

- A **don’t-care condition** is an input combination whose output is unspecified
- Either the input never occurs in the system, or the application does not care what the output is
- Example: BCD only uses 0000 through 1001; patterns 1010 to 1111 are **invalid BCD digits** that a correct system never produces
- A circuit that processes BCD digits can treat those six patterns as don’t-cares
- Don’t-cares are marked **X** (or sometimes d), distinct from 0 and 1

Using don’t-cares in a K-map

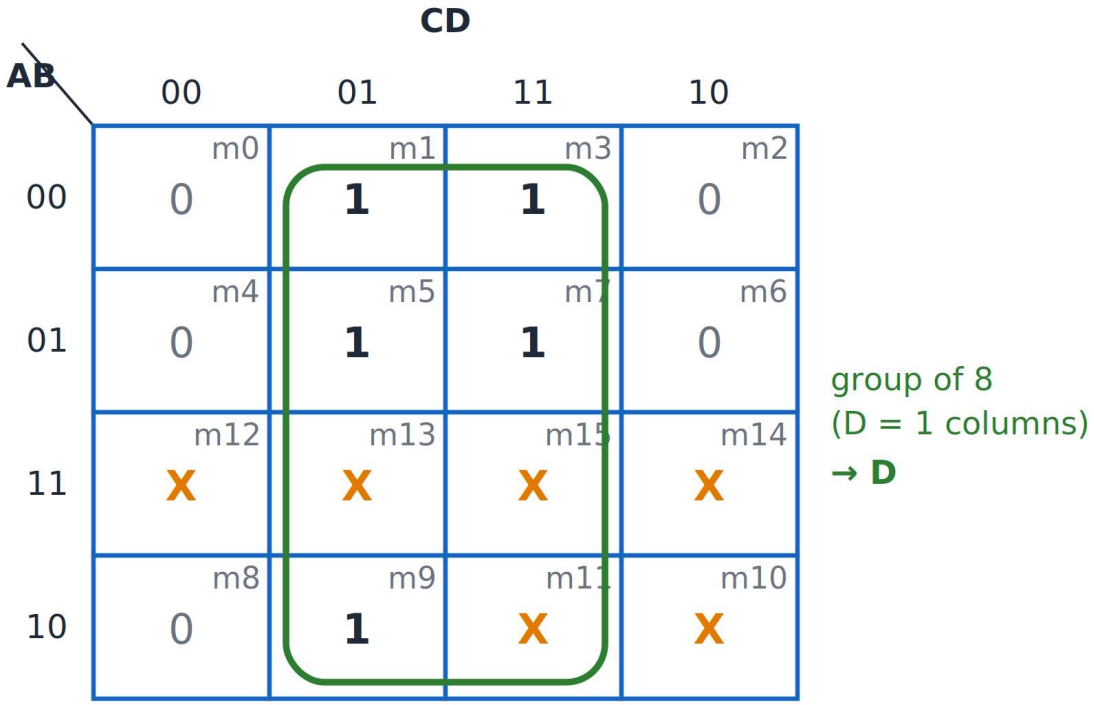
- When forming a group, treat each X as **either a 0 or a 1**, whichever gives a larger, more useful group
- Decide this **independently for each group**
- An X does not have to be in any group at all: it is optional
- Use an X only where it makes the result better

Worked example: setting up the function

$$F(A,B,C,D) = \Sigma m(1,3,5,7,9) + d(10,11,12,13,14,15)$$

- F is 1 for the odd BCD digits 1, 3, 5, 7, 9
- F is 0 for the even BCD digits 0, 2, 4, 6, 8
- The six invalid patterns (10–15) are marked as don’t-cares

K-map with don’t-cares



F = D (don’t-cares 11, 13, 15 used; 10, 12, 14 not)

- With don’t-cares 11, 13, 15 taken as 1s, the two D = 1 columns form one group of eight
- The group is {m1, m3, m5, m7, m9, d11, d13, d15}
- The term is **D**: A, B and C all vary
- d10, d12 and d14 are not needed and are left as 0

With don’t-cares 11, 13, 15 taken as 1s, the two D = 1 columns form one group of eight: F = D.

Worked example: result


```
Without don't-cares:  m9 can only pair with m1  →  F = A'D + B'C'D
With don't-cares:     treat m11, m13, m15 as 1s
Group of 8 {m1, m3, m5, m7, m9, d11, d13, d15}
F = D
```

- In the group D = 1 and A, B, C all vary
- d10, d12 and d14 are not needed and are left as 0
- The function collapses to a **single variable**: a dramatically simpler circuit

Verifying, and don’t-cares in POS

- **Verify:** F = D must equal 0 for every specified 0 (even digits 0, 2, 4, 6, 8: D = 0 for all) and 1 for every specified 1 (odd digits 1, 3, 5, 7, 9: D = 1 for all)
- The don’t-care cells (10–15) take whatever value D has there: since those inputs never occur, it does not matter
- The same freedom applies to **POS**: each X can independently be treated as a 0 (joining a 0-group) or a 1 (excluded)
- Choose whichever produces the largest, most useful 0-groups

2.5 NAND/NOR Implementation

IN SIMPLE TERMS

Any AND-OR circuit can be built from NAND gates alone, and any OR-AND circuit from NOR gates alone: just swap every gate.

SOP → NAND-NAND: the two-level conversion

- For a two-level SOP circuit (AND gates feeding a single OR gate): **replace every gate, both the AND gates and the final OR gate, with a NAND gate**
- Keep all the same connections
- The circuit computes the exact same function
- No algebra is required: it is just a wiring pattern

Why it works: DeMorgan's law

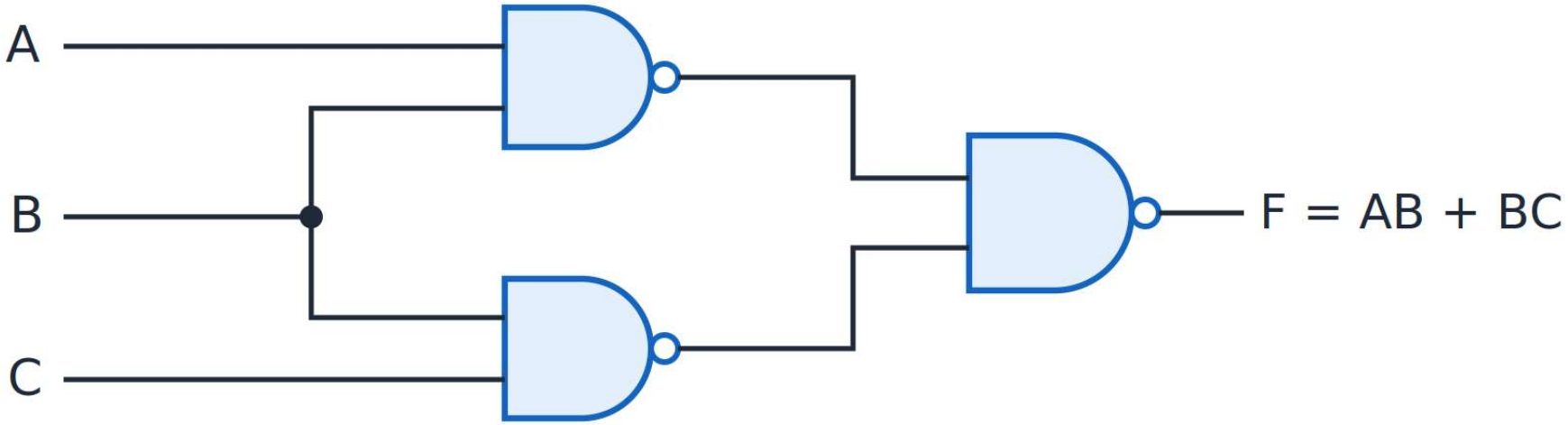

```
F = XY + ZW
Replace every gate with NAND:
((XY)' · (ZW)')' = XY + ZW
```

- The equality is DeMorgan's law, (A'B')' = A + B, with A = XY and B = ZW
- The double inversion at the second level cancels, leaving the original SOP function

Worked example: F = AB + BC

- Two first-level NAND gates compute **(AB)′** and **(BC)′**
- B fans out to both gates: a shared input feeding two gates is completely normal
- The final NAND combines them: ((AB)′ · (BC)′)′ = AB + BC
- Every gate in the circuit is a NAND: nothing else

Two-level NAND implementation of F = AB + BC



Two-level NAND implementation of F = AB + BC.

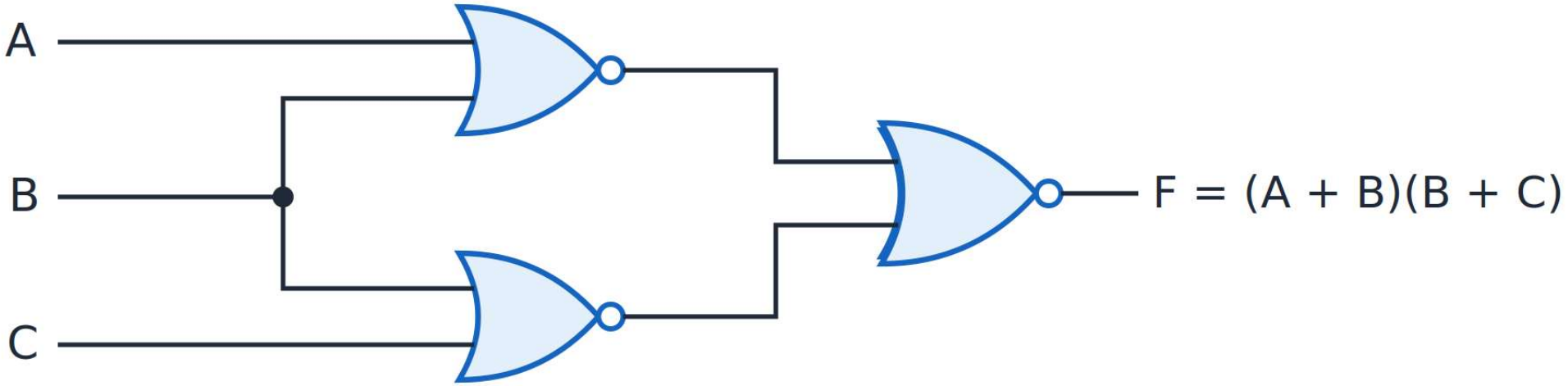
POS → NOR-NOR: the mirror conversion

- The same idea, dualised: for a two-level POS circuit (OR gates feeding a single AND gate), **replace every gate with a NOR gate**
- The justification is the dual DeMorgan law: **(A’ + B’)’ = AB**
- The circuit again computes exactly the same function

Worked example: F = (A + B)(B + C)

- Two first-level NOR gates compute **(A + B)′** and **(B + C)′**
- The final NOR combines them: ((A + B)′ + (B + C)′)′ = (A + B)(B + C)
- This is the same DeMorgan pattern as before, dualised
- Every gate is a NOR: nothing else

Two-level NOR implementation of F = (A + B)(B + C)



Two-level NOR implementation of F = (A + B)(B + C).

Unit 2 summary

- **2.1 K-maps (2 and 3 variables):** Gray-code adjacency; group 1s in powers of 2; fewest, largest groups
- **2.2 4-variable K-maps:** the edges wrap in two directions, so the four corners are neighbours
- **2.3 POS simplification:** group the 0s; variables that are 1 in the group are complemented in the sum term
- **2.4 Don’t-cares:** treat each X as 0 or 1, whichever enlarges a group
- **2.5 NAND/NOR implementation:** replace every gate in a two-level SOP with NAND, or in a POS with NOR

Review questions (1 of 2)

- **Q1.** Why are K-map headers written in Gray code order, and what does adjacency mean?
- **Q2.** Simplify F(A,B,C) = Σm(0,1,2,3,5) with a K-map
- **Q3.** Simplify F(A,B,C,D) = Σm(0,2,5,7,8,10,13,15) and explain the corner group
- **Q4.** State the rules for valid groups

Review questions (2 of 2)

- **Q5.** Find the minimal POS of F = Σm(0,1,2,3,5) by grouping the 0s
- **Q6.** Simplify F = Σm(1,3,5,7,9) + d(10,11,12,13,14,15)
- **Q7.** Implement F = AB + BC with NAND gates only
- **Q8.** Implement F = (A + B)(B + C) with NOR gates only

References

- **Textbook:** M. Morris Mano and Michael D. Ciletti, Digital Design
- **Adapted figures:** none of the figures on these pages carries an “Adapted from” credit